

AWS 3-Tier Tasks App

Implementation recap: everything built, step by step

Sebastien Ngembou · github.com/ngems1/3tier-app · September 28, 2026

The project in one paragraph

A production-style three-tier web application (React on Nginx, a FastAPI REST API, and an Amazon RDS MySQL database) deployed on AWS in two ways from one codebase: **EC2 Auto Scaling groups** with AMIs baked by Packer, and **ECS Fargate** with Docker images from ECR. All infrastructure is Terraform, and a GitHub Actions pipeline takes every change from a pull request to production with security gates, smoke tests, automatic rollback and an approval before prod. AWS access uses GitHub OIDC only: no AWS keys are stored in GitHub.

4

environments

2

platforms

9

workflows

0

stored AWS keys

How to read this document

The work is grouped into five phases, in the order it was built. Each step says what was implemented, how it works, and why it matters. The last pages list the current state of the pipeline and what is still to do.

Phase	Focus
1. Foundation	Application, Terraform infrastructure, bootstrap, EC2 and ECS pipelines
2. First deployments	Fixes found while deploying for real (ECR scans, VPC quota, database TLS)
3. Pipeline upgrades	GitHub Flow, rollback, Slack, security scans, destroy guard
4. Refinement and operations	Workflow structure, drift check, HTTPS, documentation
5. Testing and delivery	Pull request flow, ruleset, ECS test, presentation, access guides

PHASE 1

Foundation

#	What was implemented	How it works	Why it matters
1	Three-tier application	React tasks page served by Nginx; FastAPI REST API (/api/tasks, /healthz, /api/healthz); MySQL schema created by the API.	A real app with a database to deploy, test and monitor.
2	Terraform modules	Reusable modules: network (VPC, 3 AZs, NAT, flow logs), ALB with WAF, compute (ASGs), database (RDS, KMS), observability, DNS/ACM, ECS stack.	All infrastructure as code, identical across environments.
3	Four environments	EC2 dev and prod, ECS dev and prod; each with its own Terraform state, subnets, load balancers and database.	Deploying one never affects another.
4	Bootstrap workflow	Creates the S3 state bucket, the GitHub OIDC roles (plan, build, deploy-dev, deploy-prod), the artifact bucket and ECR repositories. Role names follow the cloudbatch818 rule.	Secure, repeatable account setup with no stored keys.
5	EC2 pipeline (Project 1)	CI, then build release: versioned artifacts in S3 and Packer AMIs tagged with the commit SHA; plan and apply dev, then prod after approval.	Immutable releases that can be redeployed or rolled back.
6	ECS pipeline (Project 2) and DEPLOY_TARGET	Same flow with Docker images in ECR and Fargate services. The DEPLOY_TARGET variable (ec2, ecs, both) chooses where pushes deploy.	Compare VM and container deployment of the same app.
7	Destroy workflow	Removes one environment with typed confirmation; prod needs approval and keeps a final database snapshot.	Controls cost safely.
8	Security hardening	Checkov findings fixed or justified, slimmer and upgraded Docker images, OIDC trust accepting ID-based subject claims.	Clean security scans from the start.

PHASE 2

First deployments: problems found and fixed

#	What was implemented	How it works	Why it matters
9	ECR scan gate	The pipeline now polls the image scan result, starts a scan if AWS hasn't, and supports enhanced scanning. Build role got <code>ecr:StartImageScan</code> .	No more ScanNotFound failures; CRITICAL image findings still block.
10	Existing VPC support	The account hit its VPC quota. Stacks can now be built inside an existing VPC (<code>existing_vpc_id</code>), each with its own /24 subnet ranges.	Unblocked deployments without admin quota changes.
11	Database connection hardening	RDS parameters <code>skip_name_resolve</code> and a higher <code>max_connect_errors</code> , automatic reboot when parameters are pending, and the API retries the database in the background instead of crashing.	Fixed '502' and 'host blocked' errors at startup.
12	TLS to RDS fixed	Root cause: PyMySQL ignored the CA bundle when passed as a dict. The API now passes <code>ssl_ca</code> with certificate and host verification; unit tests cover it.	Encrypted and verified database connections.

PHASE 3

Pipeline upgrades

#	What was implemented	How it works	Why it matters
13	GitHub Flow	Pull request template, a branch ruleset on main (PR required, required checks, no force push, squash merge) and a setup guide.	Every change is reviewed and checked before main.
14	Manual approval gate	A CI job in the pr-approval environment that waits for your approval after all checks pass.	Lets you approve your own PRs, which GitHub reviews don't allow.
15	Automatic rollback	If the rollout or smoke tests fail, the pipeline redeploys the last good release (recorded in SSM) and re-runs the smoke tests.	A bad release never stays in place and never reaches prod.
16	Docs-only changes skip deploys	Deploy workflows ignore changes to *.md, docs/, LICENSE and git config files.	No 30-minute deploy for a README edit.
17	Reliable tool installs	A shared setup-hashicorp action downloads Terraform and Packer with retries and SHA256 checksum verification.	Fewer random pipeline failures.
18	Slack notifications	Deploy results, prod waiting for approval (with the plan summary), rollbacks, early failures and Destroy results; approvals from Slack with the GitHub app.	You know what happened without watching GitHub.
19	Each platform builds only what it runs	Deploy EC2 builds only AMIs; Deploy ECS builds the images (also for a rollback to a release that only ran on EC2).	Faster runs and fewer moving parts.
20	CloudWatch alarms to Slack	A small Lambda per stack forwards alarms from SNS to Slack; the webhook is kept in an encrypted SSM parameter written by the pipeline.	Production problems reach you immediately.
21	Scoped Lambda permissions	Checkov flagged a broad permission; the deploy role now manages only the project's own Lambda functions.	Least privilege kept.
22	Trivy scan of every AMI	Packer scans each image before saving it and fails on fixable CRITICAL vulnerabilities. Cache moved off /tmp (a small RAM disk on Amazon Linux 2023).	Vulnerable images never become AMIs.
23	Destroy guard	Before any apply, the plan is checked: deleting or replacing the database, a load balancer or a KMS key stops the deploy (warning on PRs; override with allow_destructive).	Protects data from accidental Terraform changes.

PHASE 4

Refinement and operations

#	What was implemented	How it works	Why it matters
24	AMI scan covers Java libraries	Trivy's Java vulnerability database is downloaded (with retries) before the scan.	Fixed the AMI build failure and widened the scan.
25	Workflow structure decided	Tried a single Deploy dispatcher and an ECS-manual-only variant; kept separate Deploy EC2 and Deploy ECS workflows chosen by DEPLOY_TARGET.	Clear, flat pipeline graphs you are comfortable with.
26	Faster pull requests	The PR plan preview runs alongside the other checks instead of after Terraform.	Quicker feedback on every PR.
27	Destroy works after interrupted deploys	The deletion-protection step runs only in prod, so Destroy no longer tries to recreate resources missing from the state.	Recovery from a stopped deploy is possible.
28	Nightly drift check	Every morning a read-only plan of each deployed environment; changes made outside Terraform (or not yet deployed) turn the run red and alert Slack.	Manual console changes are caught within a day.
29	HTTPS on a custom domain	sebngembou-cloud.click configured for all four environments: ACM certificate, TLS 1.3, HTTP redirected to HTTPS, smoke tests check both.	Encrypted traffic on a real domain (live once the hosted zone exists and dev is redeployed).
30	Documentation	README (architecture, pipeline, Destroy, security, recent improvements) and runbook (drift check, rollback, recovery after an interrupted run, database access).	Anyone can understand and operate the project.

PHASE 5

Testing and delivery

#	What was implemented	How it works	Why it matters
31	Ruleset enforced and PR flow tested	protect-main switched on; a test PR ran all checks, the plan previews and the Manual approval, then merged. A code change then deployed through Deploy EC2.	The full GitHub Flow is proven end to end.
32	Production only from main	Verified three independent locks: IAM trust policies, the prod environment branch rule (main only, no admin bypass) and a required approval.	Feature branches can never reach prod.
33	Environment variables checked	The prod deploy role variable renamed to AWS_DEPLOY_ROLE_ARN; DEPLOY_TARGET documented for ECS deploys on merge.	Prod applies can log in to AWS.
34	ECS pull request test prepared	Step-by-step flow: set DEPLOY_TARGET, change the heading on a branch, PR, approve, merge, watch Deploy ECS.	Both platforms validated through the same process.
35	Presentation and guides	An 11-slide PowerPoint with speaker notes, and guides for viewing database data (API or SQL through Session Manager).	Ready to present and demo the project.

Current state of the pipeline

Workflow	Trigger	What it does
CI (ci.yml)	Every pull request; called by the deploys	Lint, tests, Terraform checks, Checkov, Trivy, CodeQL, audits; on PRs also plan previews and Manual approval
Deploy EC2	Push to main (DEPLOY_TARGET ec2 or both) or manual	AMIs + Trivy, dev with smoke tests and rollback, prod after approval
Deploy ECS	Push to main (DEPLOY_TARGET ecs or both) or manual	Images + scans, ecs-dev with smoke tests and rollback, ecs-prod after approval
Drift check	Every morning or manual	Read-only plan of every deployed environment; Slack alert on differences
Bootstrap	Manual	Account setup: state bucket, OIDC roles, artifact bucket, ECR
Destroy	Manual	Removes one environment with typed confirmation (prod needs approval)
Reusable workflows	Called by the others	Terraform plan, Terraform apply (with rollback), container images

Still to do

- **HTTPS go-live:** confirm the hosted zone for sebgembou-cloud.click exists, finish the dev cleanup, then deploy.
- **ECS test:** set DEPLOY_TARGET to ecs or both and run the ECS pull request test.
- **Quick wins:** pin GitHub Actions to commit SHAs with Dependabot, AWS Budgets alert, pin runners to ubuntu-24.04, scan results in the Security tab.
- **Cost and reliability:** stop dev at night, rebuild only what changed, a Terraform maintenance workflow (unlock/import), uptime checks, AWS Backup for prod.
- **Bigger projects:** blue/green or canary deployments, database migrations with Alembic, load tests with k6.